

NetOps AI — Requirements Document

1. Product Requirements

What problem does the project solve?

The project solves the problem of **manual, slow, and reactive network operations**.

Network teams usually need to:

- monitor many routers/switches/servers manually
- detect failures late
- troubleshoot using logs/CLI
- handle configuration drift manually
- generate reports manually
- recover systems after downtime

NetOps AI centralizes monitoring, automation, configuration validation, alerting, and reporting in one platform, similar to the centralized network management concept described for Cisco Catalyst Center.

Who are the target users?

User Type	Description
Network Admins	Manage routers, switches, APs, edge nodes
NOC Engineers	Monitor live health, alerts, incidents
SRE / DevOps Engineers	Track reliability, uptime, automation workflows
IT Managers	View reports, SLA, incident trends
Intern/Junior Engineers	Build/debug Python automation and monitoring features

Main use cases

1. **Monitor network devices in real time**
2. **Detect latency, packet loss, CPU/memory issues**

- 3. **Detect device offline/heartbeat failure**
- 4. **Create alerts and incidents automatically**
- 5. **Detect configuration drift**
- 6. **Trigger automated recovery workflow**
- 7. **Generate uptime/SLA/config compliance reports**
- 8. **View live NOC dashboard**

Mandatory MVP features

MVP Feature	Required
Device simulator	Yes
Device inventory	Yes
Real-time telemetry ingestion	Yes
Dashboard overview	Yes
Alerts system	Yes
Incident creation	Yes
Config drift detection	Yes
Basic automation workflow	Yes
REST APIs	Yes
WebSocket live updates	Yes
PostgreSQL/TimescaleDB storage	Yes

Later features

Future Feature	Purpose
AI anomaly detection	Predict unusual behavior
LLM incident summaries	Explain root cause in simple language
PDF reports	Professional reporting

Role-based access control	Admin/user permissions
SNMP simulation	More realistic network protocol
NetFlow simulation	Traffic analytics
Grafana/Prometheus	Observability
Kubernetes deployment	Production scalability

2. Functional Requirements

User roles

Role	Permissions
Admin	Full access, manage devices, configs, users, automation
Engineer/User	View devices, alerts, incidents, run diagnostics
Guest/Viewer	Read-only dashboard access

Inputs

Input Type	Example
Simulated telemetry	CPU, memory, latency, packet loss
Real machine metrics	psutil CPU/memory/network I/O
Config files/forms	YAML/JSON device configuration
API requests	Device registration, automation trigger
WebSocket events	Live telemetry and alerts
Optional datasets	ML anomaly detection training

Outputs

Output	Description
Dashboard	Device health, topology, alerts
Alerts	Warning/critical events
Incidents	Tracked failure records
Reports	Uptime, SLA, config compliance
Predictions	Anomaly/failure risk
Emails/Webhooks	Alert notifications
Automation logs	Recovery workflow results

CRUD operations needed

Devices

- Create device
- View devices
- Update metadata
- Delete device

Configs

- Create config
- View current/baseline config
- Update config
- Rollback config

Incidents

- Create incident
- View incidents
- Update status
- Resolve incident

Alerts

- Create alert automatically
- View alerts
- Update alert status

Automation workflows

- Create workflow
 - Run workflow
 - View run history
 - Update workflow rules
-

Authentication/login required?

For MVP: **optional but recommended**.

Best plan:

Phase	Auth
MVP	Simple JWT login
Advanced	RBAC with admin/user/guest
Production-style	OAuth/SSO-ready structure

3. Non-Functional Requirements

Real-time or batch processing?

This project should be **real-time first**.

Processing Type	Usage
Real-time	Telemetry, alerts, device status, incidents
Batch	Daily reports, SLA summaries, anomaly model training

Telemetry should stream every:

2–5 seconds per device

Expected users/traffic

For project MVP:

Metric	Target
Simulated devices	50–100
Concurrent dashboard users	5–20
Telemetry interval	2–5 seconds
Events/minute	600–3000
WebSocket clients	5–20

Advanced version:

500+ devices
10,000+ telemetry events/minute

Performance expectations

Requirement	Target
API response time	< 300 ms
Dashboard live update delay	< 2 seconds
Alert creation delay	< 3 seconds
Telemetry ingestion	Near real-time
Report generation	< 10 seconds
Device health calculation	< 1 second

Security needs

Required security:

- JWT authentication
- role-based access control
- password hashing
- input validation
- API rate limiting
- audit logs
- environment variables for secrets
- secure config storage
- CORS restriction

Advanced security:

- encrypted config values
 - admin approval for rollback
 - audit trail for automation actions
 - OAuth/SSO integration
-

Reliability/uptime expectations

For project version:

95–99% local/demo uptime

Production-style target:

99.9% service availability

Reliability features:

- retry failed jobs
- health check endpoints
- Redis queue persistence
- failed event logging
- worker restart support
- fallback alert processing
- database backups

Scalability requirements

MVP should use:

Modular FastAPI backend

- + Redis Streams
- + PostgreSQL/TimescaleDB
- + Celery workers
- + WebSockets

Scalable future design:

Split services:

- telemetry-service
- device-service
- config-service
- automation-service
- incident-service
- reporting-service

Scaling strategy:

Component	Scaling Method
API backend	Horizontal replicas
Telemetry workers	Multiple consumers
Redis/Kafka	Stream partitioning
TimescaleDB	Hypertables/indexing
WebSockets	Redis pub/sub
Reports	Async background jobs

Final One-Line Requirement Summary

NetOps AI is a real-time Python-based network monitoring and automation platform that simulates enterprise devices, streams telemetry, detects failures, manages configurations, automates recovery, and provides dashboards, alerts, incidents, and reports for network operations teams.